Smart Retail Inventory & Billing System

Complete Technical Architecture, Function Guide & Viva Examination Manual

A full-scale desktop Point-of-Sale (POS) and inventory management platform built entirely with pure Java (JDK 17+), Java Swing, JDBC, and MySQL 8.x, enforcing MVC separation, ACID transactions, and Role-Based Access Control.

LANGUAGE	UI FRAMEWORK	DATABASE ENGINE	ARCHITECTURE
Java (JDK 17+)	Java Swing & Java 2D	MySQL 8.x (JDBC)	Strict MVC & DAO

1. System Overview & MVC Architecture

The application follows the industry-standard **Model-View-Controller (MVC)** architectural pattern paired with the **Data Access Object (DAO)** design pattern. This architecture completely decouples visual presentation from business logic and database persistence, preventing spaghetti code and eliminating SQL queries from Swing UI action listeners.

Architectural Layer Flow

User Interaction Flow:

```
[ User / Cashier / Admin ]
    | (Mouse clicks, barcode scanning, keyboard input)
    ▼
VIEW LAYER (Java Swing: Frames, Panels, Tables, Dialogs)
    | (Fires Controller action methods; receives rendered DTO models)
    ▼
CONTROLLER LAYER (Orchestrators: BillingController, ProductController, etc.)
    | (Executes validation, orchestrates transactions, calculates math)
    ▼
MODEL LAYER (Pure POJOs: Product, Bill, BillItem, User, Inventory)
    | (Represents state and domain business constraints)
    ▼
DAO LAYER (Data Access Objects: BillDAO, ProductDAO, UserDAO)
    | (PreparedStatement queries, connection lifecycle, ACID rollback)
    ▼
DATABASE ENGINE (MySQL 8.x on Port 3306 via Connector/J)
```

2. Complete Class-by-Class Reference

The codebase is organized into cleanly separated packages under `src/`:

A. Domain Models (`src/model/`)

Class Name	Extends / Implements	Responsibility & Business Logic
`BaseEntity`	*Abstract Base*	Common foundation for all models. Encapsulates `id`, `createdAt`, and `updatedAt` timestamps with strict getters and setters.
`User`	`BaseEntity`	Represents system operators. Stores `username`, `password`, `fullName`, `role` (`ADMIN` or `CASHIER`), and `isActive` status.
`Product`	`BaseEntity`	Catalog domain object. Stores `barcode`, `name`, `category`, `costPrice`, `sellingPrice`, `stockQuantity`, and `minStockAlert`. Validates non-negative prices.
`Inventory`	`BaseEntity`	Warehouse monitoring model. Calculates stock deficits and evaluates badge health (`OK`, `LOW STOCK`, `OUT OF STOCK`).
`Bill`	`BaseEntity`	Invoice transaction header. Encapsulates `billNumber`, `cashierId`, `customerPhone`, `subtotal`, `taxRate` (8%), `taxAmount`, `grandTotal`, `paymentMethod`, and the list of line items.
`BillItem`	`BaseEntity`	Individual invoice line in a cart. Tracks `productId`, `productName`, `quantity`, `unitPrice`, and calculated `totalPrice`.
`SalesReport`	—	Aggregated metric container used for executive KPIs and charts (gross revenue, transaction volume, best-selling products).

B. Data Access Objects (`src/dao/`)

Class Name	Extends / Implements	Database Responsibilities
`BaseDAO`	*Abstract Base*	Acquires JDBC connections from `DatabaseConnection`. Implements centralized, safe resource cleanup for `ResultSet`, `Statement`, and `Connection`.
`UserDAO`	`BaseDAO`	Authenticates operators with `SELECT * FROM users WHERE username = ? AND password = ? AND is_active = 1`. Handles staff CRUD.
`ProductDAO`	`BaseDAO`	CRUD operations for products, live barcode scanner lookup, keyword search, and low-stock filtering queries.
`BillDAO`	`BaseDAO`	**Executes ACID checkout transactions**. Manages `conn.setAutoCommit(false)`, inserts bill headers and item details, deducts stock atomically, and executes `conn.rollback()` upon any failure.
`ReportDAO`	`BaseDAO`	Runs aggregate SQL queries (`SUM`, `COUNT`, `DATE_SUB`, `GROUP BY`) for today's sales, weekly trends, and historical monthly summaries.

C. Controllers (`src/controller/`)

Class Name	Workflow & Orchestration Responsibility
`LoginController`	Validates credentials, invokes `UserDAO`, stores the authenticated session, and routes the user to the Admin or Cashier dashboard.
`ProductController`	Validates catalog form inputs, delegates CRUD operations to `ProductDAO`, and refreshes the product table.
`InventoryController`	Handles restock requests, applies stock threshold rules, and updates inventory monitors.
`BillingController`	Core POS engine. Manages cart state, validates live stock prior to adding items, computes Subtotal + 8% GST + Grand Total, and triggers atomic checkout.
`ReportController`	Retrieves analytics data and feeds it to KPI cards and the custom Java 2D bar chart.
`UserController`	Manages staff account creation, role assignments, and status toggles.

D. View Layer (`src/view/`)

Class Name	Swing Component	UI Responsibilities
`LoginFrame`	`JFrame`	Centered authentication window with custom styled inputs, password masking, and role validation feedback.
`AdminDashboard`	`JFrame`	Manager portal featuring a collapsible sidebar drawer, live operator profile display, and `CardLayout` navigation.
`CashierDashboard`	`JFrame`	Streamlined station for billing operators, locked exclusively to the POS billing module.
`DashboardHomePanel`	`JPanel`	Executive overview showing 4 KPI cards (Daily Sales, Invoices, Low Stock Alerts, Total Products) and recent orders.
`ProductPanel`	`JPanel`	Catalog manager with live search filtering, pagination/tables, and add/edit forms.
`InventoryPanel`	`JPanel`	Stock health dashboard with low-stock quick filters and a quick-restock modal dialog.
`BillingPanel`	`JPanel`	Two-column POS terminal: live catalog search on left, shopping cart and payment summary on right.
`ReceiptDialog`	`JDialog`	Simulated 58mm thermal receipt preview modal with PDF and TXT export actions.
`ReportsPanel`	`JPanel`	Analytics screen supporting daily, weekly, and monthly views with an embedded Java 2D chart.
`SalesBarChartPanel`	`JPanel`	Custom component overriding `paintComponent(Graphics g)` to draw an anti-aliased 2D vertical bar chart.
`StockStatusCellRenderer`	`TableCellRenderer`	Custom renderer painting color-coded badges for stock health inside table cells.
`UserPanel`	`JPanel`	Staff administration panel for managing system operators.

E. Utilities & Infrastructure (`src/util/`)

Class Name	Design Pattern / Purpose
`DatabaseConnection`	**Singleton Pattern**. Manages JDBC connections, auto-creates the MySQL database on port 3306, and bootstraps schema and seed data.
`UIStyleUtil`	**Factory Pattern**. Enforces the Coolors palette (`#04151F` , `#183A37` , `#EFD6AC` , `#C44900`) across buttons, tables, text fields, and cards.
`ValidationUtil`	Static validation helpers for currency, quantities, barcodes, phone numbers, and mandatory inputs.
`ReceiptExporter`	**Strategy Pattern**. Defines an interface with two implementations: `TextExporter` (.txt) and a pure Java `PdfExporter` (.pdf) without third-party dependencies.

3. How Each Core Function Operates (Java Workflows)

1. Self-Healing Bootstrapping & Automatic Schema Setup

When `Main.main()` executes, it triggers `DatabaseConnection.initializeDatabase()`. The method connects to MySQL root (`jdbc:mysql://localhost:3306/`) without targeting a database name. It executes `CREATE DATABASE IF NOT EXISTS smart_retail`, switches the JDBC connection URL to `jdbc:mysql://localhost:3306/smart_retail`, and builds tables from `resources/schema.sql`. If the tables are newly initialized, it inserts default administrative credentials and sample catalog items.

2. Role-Based Access Control (RBAC) Routing

Upon submitting credentials in `LoginFrame`, `LoginController.handleLogin()` calls `UserDAO.authenticate(user, pass)`. If valid, the returned `User` instance is inspected for role authorization. If `ADMIN`, the full `AdminDashboard` loads; if `CASHIER`, the restricted `CashierDashboard` launches. Administrative buttons and navigation panels are not rendered for cashier accounts, enforcing defense-in-depth.

3. POS Cart Operations & Real-time Tax Mathematics

When a barcode is entered or a product is selected in `BillingPanel`, `BillingController.addItemToCart()` queries the active catalog. If `currentStock < requestedQuantity`, an error dialog blocks addition. Otherwise, the item is added to the cart's `DefaultTableModel`. On every modification, `calculateTotals()` executes:

```
Subtotal    = Σ (selling_price × quantity)
Tax (8% GST) = Subtotal × 0.08
Grand Total = Subtotal + Tax
```

4. ACID Transaction Checkout with Rollback Protection

The most critical method in the system is `BillDAO.createBillWithItems()`. It executes all checkout steps within an explicit atomic transaction:

```
Connection conn = DatabaseConnection.getConnection();
conn.setAutoCommit(false); // Enable ACID manual transaction mode
try {
    // Phase 1: Re-verify stock lock for every item
    for (BillItem item : bill.getItems()) {
        int liveStock = queryCurrentStock(conn, item.getProductId());
        if (liveStock < item.getQuantity()) {
            throw new SQLException("Insufficient stock: " + item.getProductName());
        }
    }
    // Phase 2: Insert invoice header into `bills`
    int billId = executeInsertBill(conn, bill);

    // Phase 3: Insert items into `bill_items` and deduct product inventory
    for (BillItem item : bill.getItems()) {
        executeInsertBillItem(conn, billId, item);
        executeDecrementStock(conn, item.getProductId(), item.getQuantity());
    }

    conn.commit(); // Transaction succeeds: all modifications applied simultaneously
} catch (Exception ex) {
    conn.rollback(); // Transaction failed: every insert and stock deduction is completely un
    throw ex;
} finally {
    conn.setAutoCommit(true); // Restore default connection state
}
```

5. Custom Java 2D Graphics Engine (Sales Bar Chart)

Rather than requiring heavy external libraries like JFreeChart, `SalesBarChartPanel` extends `JPanel` and overrides `paintComponent(Graphics g)`. It casts to `Graphics2D`, turns on `RenderingHints.KEY_ANTIALIASING`, determines the highest sales value in the dataset, dynamically computes coordinate scaling ratios, draws the axes and grid lines, and renders vertical rectangles colored with `#C44900` (Rust) and `#183A37` (Pine) with numeric text tags centered above each bar.

4. Comprehensive Viva Examination Questions & Answers

Category 1: Object-Oriented Programming (OOP) Principles

Q How is Encapsulation demonstrated in this project?

Encapsulation is implemented by keeping all class member variables `private` across domain models (e.g., `Product`, `User`, `Bill`) and providing controlled access via public getters and setters. Setters enforce validation logic—for example, `Product.setSellingPrice(double price)` checks that the price is strictly positive, preventing invalid state mutations.

Q Where is Inheritance used, and what architectural benefit does it provide?

Inheritance is used in two key hierarchies:

- `BaseEntity`: Inherited by `User`, `Product`, `Bill`, and `BillItem`. It provides shared fields (`id`, `createdAt`, `updatedAt`) and standardizes lifecycle metadata.
- `BaseDAO`: Inherited by `UserDAO`, `ProductDAO`, `BillDAO`, and `ReportDAO`. It provides centralized connection management and safe resource teardown methods (`close()`).

This avoids code duplication and enforces structural consistency across the application.

Q How does your project use Polymorphism?

Polymorphism is demonstrated via the **Strategy Pattern** in `ReceiptExporter`. The interface defines `File export(Bill bill, File dir)`, which is implemented polymorphically by `TextExporter` and `PdfExporter`. The calling view invokes the shared interface method without needing to know which exporter format is active. Polymorphism is also utilized in `StockStatusCellRenderer`, which overrides Swing's `getTableCellRendererComponent()` to dynamically customize cell styling based on row data.

Q How is Abstraction implemented in the persistence layer?

Abstraction hides complex low-level details. The View and Controller layers do not know whether the system uses MySQL, Oracle, or files; they simply interact with DAO abstract contracts like `productDAO.findById(id)`. All JDBC connections, SQL syntax, driver loading, and `ResultSet` parsing are abstracted inside the DAO layer.

Q Why did you use the MVC pattern instead of placing database queries directly inside button action listeners?

Putting SQL queries inside Swing event listeners couples presentation directly with data storage, making the system difficult to maintain, test, or update. MVC enforces **Separation of Concerns**:

- **View** only renders GUI elements and captures user gestures.
- **Controller** processes logic, enforces validation, and handles navigation.
- **Model** holds state and business constraints.
- **DAO** manages data persistence.

This modularity allows modifying the database schema or swapping the UI without breaking business logic.

Category 2: Database, JDBC & ACID Transactions

Q What are ACID properties, and how does your project guarantee them?

ACID ensures reliable database transaction processing:

- **Atomicity:** All-or-nothing execution. In `BillDAO.createBillWithItems()`, inserting the bill header, adding cart line items, and deducting inventory stock happen together. If any operation fails, `conn.rollback()` reverts all changes.
- **Consistency:** Database constraints (foreign keys, non-negative stock checks) are preserved before and after each transaction.
- **Isolation:** MySQL's InnoDB engine ensures concurrent transactions execute independently without dirty reads.
- **Durability:** Once `conn.commit()` completes, the transaction is written to MySQL's redo log and persists across power failures or crashes.

Q Why is `conn.setAutoCommit(false)` required in billing checkout?

By default, JDBC operates in auto-commit mode (every individual SQL statement commits immediately). If the bill header were inserted and the system crashed before stock deduction, inventory counts would be corrupted. Calling `conn.setAutoCommit(false)` groups all operations into a single logical transaction, ensuring manual control over `commit()` or `rollback()`.

Q How does your code prevent SQL Injection attacks?

Every query throughout all DAOs strictly utilizes `java.sql.PreparedStatement` with parameterized placeholders (`?`). User input is never concatenated directly into SQL strings. The JDBC driver treats inputs as literal data values rather than executable SQL commands, neutralizing injection attempts.

Q Explain the database schema and foreign key relationships.

The relational schema consists of 4 core tables:

- `users`: Primary key `id`. Stores authentication and roles.
- `products`: Primary key `id`, unique `barcode`. Stores prices and inventory quantities.
- `bills`: Primary key `id`, unique `bill_number`, foreign key `cashier_id REFERENCES users(id)`.
- `bill_items`: Primary key `id`, foreign key `bill_id REFERENCES bills(id) ON DELETE CASCADE`, foreign key `product_id REFERENCES products(id)`.

Q How does the application handle database creation if MySQL is fresh?

`DatabaseConnection.java` uses a two-phase connection strategy. It first connects without a database name (`jdbc:mysql://localhost:3306/`) to execute `CREATE DATABASE IF NOT EXISTS smart_retail`. It then connects directly to `smart_retail` and executes table creation scripts and initial seed data if the tables are empty.

Category 3: Java Swing, UI & Event Handling

Q What is the Event Dispatch Thread (EDT) and why is `SwingUtilities.invokeLater()` used?

Java Swing is **single-threaded** and not thread-safe. All component creation, updates, and event listeners must run on the **Event Dispatch Thread (EDT)**. `SwingUtilities.invokeLater()` places the UI initialization runnable at the end of the AWT event queue, preventing race conditions, UI freezing, and visual rendering glitches.

Q How did you create the custom 2D Bar Chart without external libraries?

In `SalesBarChartPanel`, the standard `paintComponent(Graphics g)` method was overridden. The `Graphics` object is cast to `Graphics2D` to enable anti-aliasing. The panel calculates the maximum revenue among the data points, normalizes each bar's height relative to the panel's pixel height, draws coordinate grid lines, and renders colored rectangles for each date using the system's Rust and Pine palette.

Q How does `StockStatusCellRenderer` work for table status badges?

It extends `DefaultTableCellRenderer`. During table painting, Swing invokes `getTableCellRendererComponent()` for every cell. The renderer inspects the stock quantity: if `qty ≤ 0`, it renders an "OUT OF STOCK" badge in soft red; if `qty ≤ minAlert`, it renders "LOW STOCK" in amber; otherwise, it renders "OK" in green with rounded borders and bold text.

Q Which layout managers are used and why?

A combination of layouts is used based on view requirements:

- `BorderLayout`: Used for dashboard roots (Top header in `NORTH`, Navigation in `WEST`, Main work area in `CENTER`).
- `CardLayout`: Used in `AdminDashboard` to switch between panels (Dashboard, Products, Billing, Inventory, Reports) without creating and destroying frames.
- `GridBagLayout`: Used for forms, login dialogs, and detail cards where precise column alignment and responsive resizing are required.

Category 4: Security, Business Logic & Edge Cases

Q **How is Role-Based Access Control (RBAC) enforced in the application?**

When an operator logs in, their role (`ADMIN` or `CASHIER`) is stored in the active session. If the user is a cashier, they are routed to `CashierDashboard` , which only renders the billing screen. Administrative navigation panels (Products, Stock adjustments, Staff management, Financial reports) are not instantiated for cashiers, preventing unauthorized access.

Q **What happens if a cashier tries to sell more items than available in stock?**

The application protects against this at two independent levels:

1. **UI Level:** `BillingController.addItemToCart()` checks if `requestedQuantity > availableStock` . If so, it alerts the cashier immediately and prevents cart insertion.
2. **Database Level:** Inside `BillDAO.createBillWithItems()` , before executing inserts, a live stock query re-verifies inventory. If stock is insufficient, it throws an exception and rolls back the transaction.

Q **How are edge cases like negative product prices or empty inputs handled?**

`ValidationUtil.java` validates all inputs before persistence:

- Prices must parse to positive doubles (`> 0`).
- Quantities must be positive integers.
- Barcodes must be non-empty alphanumeric strings.
- Customer phone numbers are validated against standard format patterns.

Any validation error displays a user-friendly message and leaves the database untouched.

Q **How does the PDF Receipt Exporter work without iText or Apache PDFBox?**

`ReceiptExporter.PdfExporter` writes directly to a `FileOutputStream` using standard **PDF 1.4 specification stream syntax**. It outputs the standard PDF header (`%PDF-1.4`), Catalog object, Pages tree, Font dictionary (standard Type 1 Helvetica), and a text stream (`BT ... ET`) containing invoice details and cross-reference tables (`xref`). This creates valid, printable PDF receipts with zero external JAR dependencies.

Q **What future enhancements would you suggest for this system?**

Potential production enhancements include:

- Migrating to password hashing with bcrypt or PBKDF2 with salting.
- Integrating a hardware ESC/POS thermal receipt printer driver via serial/USB.
- Adding barcode generation (Code128 / QR codes) for in-store printing.
- Adding payment gateway APIs (e.g. UPI dynamic QR code display at checkout).

---

5. Quick Viva Cheatsheet (Formulas & Key Reference)

Key Formulas Used in Code

POS Cart Calculation (BillingController):

- `LineItemTotal = UnitPrice × Quantity`
- `Subtotal = Σ LineItemTotals`
- `TaxAmount = Subtotal × 0.08 (8% GST)`
- `GrandTotal = Subtotal + TaxAmount`

Bar Chart Coordinate Mapping (SalesBarChartPanel):

- `BarHeight = (DayRevenue / MaxRevenue) × (ChartHeight − 60px)`

Essential SQL Tables & Constraints

Table	Key Columns	Constraints
`users`	id, username, password, role, is_active	PRIMARY KEY (id), UNIQUE (username)
`products`	id, barcode, name, cost_price, selling_price, stock_quantity	PRIMARY KEY (id), UNIQUE (barcode)
`bills`	id, bill_number, cashier_id, grand_total, payment_method	PRIMARY KEY (id), UNIQUE (bill_number), FK (cashier_id)
`bill_items`	id, bill_id, product_id, quantity, unit_price, total_price	PRIMARY KEY (id), FK (bill_id CASCADE), FK (product_id)

Default Credentials & Database Configuration

Role	Username	Password	Permissions
Admin	`admin`	`admin123`	Full back-office, catalog, inventory, users, financial reports
Cashier	`cashier`	`cashier123`	POS checkout station, cart management, receipt printing

MySQL Config (resources/db.properties): Host: `localhost` | Port: `3306` | Database: `smart_retail` | User: `root`